CÓDIGO FUENTE

MI DIRECTOR FINANCIERO PTY

Programa de Ordenador (Software)

Versión 1.1.0

Autora: Soraya Dayan Gutiérrez Pérez

Titular: Soraya Dayan Gutiérrez Pérez

(Persona Natural)

Panamá, marzo de 2026

Página 1 de 33

Registro ante la Dirección Nacional de Derecho de Autor

Ministerio de Cultura — República de Panamá

Página 2 de 33

TABLA DE CONTENIDO

Archivos incluidos (primeras y últimas líneas de cada uno):

1. frontend/src/app/dashboard/page.tsx (2194 líneas)
2. frontend/src/lib/calculations.ts (369 líneas)
3. frontend/src/lib/alerts.ts (557 líneas)
4. frontend/src/components/rrhh/MiRRHH.tsx (2224 líneas)
5. frontend/src/lib/ventas-storage.ts (367 líneas)
6. frontend/src/lib/analytics/cb-insights-detector.ts (628 líneas)
7. frontend/src/lib/rrhh/nomina-calculator.ts (375 líneas)
8. frontend/src/lib/analytics/forecast-engine.ts (508 líneas)
9. frontend/src/services/facturacion/FacturacionService.ts (181 líneas)
10. frontend/src/lib/reportes/reporte-engine.ts (333 líneas)

Nota:

Para cada archivo se muestran las primeras ~80 líneas (equivalente a ~3 páginas) y las últimas ~50 líneas (equivalente a ~2 páginas) del código fuente original.

Total de archivos de código fuente en el proyecto: ~185 archivos
Total de líneas de código aproximadas: ~76,000 líneas

Página 3 de 33

frontend/src/app/dashboard/page.tsx

Total: 2194 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  "use client";
2  import React, { useState, useMemo } from "react";
3  import {
4    BarChart3,
5    TrendingUp,
6    AlertTriangle,
7    DollarSign,
8    Building2,
9    History,
10   Zap,
11   Skull,
12   Shield,
13   Wind,
14   SlidersHorizontal,
15   FlaskConical,
16   Gem,
17   Scale,
18   FileText,
19   BookOpen,
20   FolderTree,
21   BookMarked,
22   CheckSquare,
23   Lock,
24   ArrowLeftRight,
25   Target,
26   FileBarChart,
27   Package,
28   ClipboardList,
29   Rocket,
30   Wallet,
31   Home,
32   ArrowLeft,
33   PenLine,
34   Bot,
35   Upload,
36   Pencil,
37   ImageIcon,
38   Percent,
39   Activity,
40   Check,
41   Landmark,
42   Users,
43   Receipt,
44 } from "lucide-react";
45 import type { FinancialRecord } from "@/lib/calculations";
46 import MidfLogo from "@/components/MidfLogo";
47 import BottomNavBar from "@/components/BottomNavBar";
48 import DataEntryWizard from "@/components/DataEntryWizard";
49 import NaturalLanguageInput from "@/components/NaturalLanguageInput";
50 import AuditTimeline from "@/components/AuditTimeline";
51 import DiagnosticCard from "@/components/DiagnosticCard";
52 import WaterfallChart from "@/components/charts/WaterfallChart";
53 import BreakevenChart from "@/components/charts/BreakevenChart";
54 import RatioGauges from "@/components/charts/RatioGauges";
55 import MandibulasChart from "@/components/charts/MandibulasChart";
56 import PayrollEngine from "@/components/PayrollEngine";
```

```
57 import LegalVault from "@/components/LegalVault";
58 import WatchdogDashboard from "@/components/WatchdogDashboard";
59 import SimuladorEstrategico from "@/components/SimuladorEstrategico";
60 import OxigenoTab from "@/components/OxigenoTab";
61 import LabPrecios from "@/components/LabPrecios";
62 import ValoracionTab from "@/components/ValoracionTab";
63 import LegalSimplifierButton from "@/components/LegalSimplifierButton";
64 import SmartTooltip from "@/components/SmartTooltip";
65 import AlertsSidebar, { AlertBellButton } from "@/components/AlertsSidebar";
66 import SetupWizard from "@/components/SetupWizard";
67 import BalanceGeneralCard from "@/components/BalanceGeneralCard";
68 import ChartOfAccounts from "@/components/accounting/ChartOfAccounts";
69 import LibroDiario from "@/components/accounting/LibroDiario";
70 import LibroMayor from "@/components/accounting/LibroMayor";
71 import BalanceComprobacion from "@/components/accounting/BalanceComprobacion";
72 import PeriodClosingPanel from "@/components/accounting/PeriodClosingPanel";
73 import LibroInventarios from "@/components/accounting/LibroInventarios";
74 import LibroActas from "@/components/accounting/LibroActas";
75 import PeriodSelector from "@/components/PeriodSelector";
76 import TrendsChart from "@/components/charts/TrendsChart";
77 import ForecastSection from "@/components/ForecastSection";
78 import ComparisonView from "@/components/charts/ComparisonView";
79 import BudgetChart from "@/components/charts/BudgetChart";
80 import BudgetEntryForm from "@/components/BudgetEntryForm";
```

... 2064 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
2145                {activeLegalTab === "libro_actas" && (<div><h2 className="text-lg lg:text-xl
font-bold text-slate-800 mb-4">Libro de Actas</h2><LibroActas societyId={societyId} /></div>)}
2146                {activeLegalTab === "traductor" && (<div><h2 className="text-lg lg:text-xl
font-bold text-indigo-800 mb-4">Traductor Legal</h2><TraductorLegal /></div>)}
2147              </div>
2148              <div className="bg-white rounded-2xl border border-slate-200 p-4 lg:p-6">
2149                <h3 className="text-sm font-bold text-slate-600 mb-3 flex items-center gap-2">
2150                  <FileText size={16} className="text-violet-500" />
2151                  Traductor Legal
2152                </h3>
2153                <p className="text-xs text-slate-400 mb-3">Pega cualquier clausula legal y te
la explicamos en espanol sencillo.</p>
2154                <LegalSimplifierButton />
2155              </div>
2156            </div>
2157          )}
2158        </div>
2159
2160        <BottomNavBar active={activeSection} onNavigate={setActiveSection} silverMode=
{isSilverMode} />
2161
2162        {/* Panel de Alertas Estrategicas (sidebar derecho) */}
2163        <AlertsSidebar
2164          alerts={strategicAlerts}
2165          isOpen={alertsSidebarOpen}
2166          onClose={() => setAlertsSidebarOpen(false)}
2167        />
2168
2169        {/* Mi Asistente — Chatbot IA Flotante */}
2170        <MiAsistente
2171          societyId={societyId}
2172          onResult={(result) => {
2173            if (result?.action === "journal_entry_created") {
2174              // Refrescar datos si se creo un asiento
```

```
          }
        }}
      />
    </main>
    </OnboardingProvider>
  );
}

function SummaryRow({ label, value, color }: { label: React.ReactNode; value: number; color: string }) {
  return (
    <div className="flex items-center justify-between">
      <span className="text-sm text-slate-500">{label}</span>
      <span className={`text-sm font-bold ${color}`}>${value?.toLocaleString("es-PA")}</span>
    </div>
  );
}

function EmptyState({ text }: { text: string }) {
  return <p className="text-slate-400 text-center py-12 text-sm">{text}</p>;
}
```


Página 6 de 33

frontend/src/lib/calculations.ts

Total: 369 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  /**
2   * Funciones de calculo financiero client-side para Mi Director Financiero PTY.
3   * Usadas por SimuladorEstrategico, LabPrecios y ValoracionTab.
4   */
5
6  // ============================================
7  // TIPOS
8  // ============================================
9
10 export interface FinancialRecord {
11   revenue: number;
12   cogs: number;
13   opex_rent: number;
14   opex_payroll: number;
15   opex_other: number;
16   depreciation: number;
17   interest_expense: number;
18   tax_expense: number;
19   cash_balance: number;
20   accounts_receivable: number;
21   inventory: number;
22   accounts_payable: number;
23   bank_debt: number;
24 }
25
26 export interface CascadaResult {
27   revenue: number;
28   cogs: number;
29   gross_profit: number;
30   total_opex: number;
31   ebitda: number;
32   ebit: number;
33   ebt: number;
34   net_income: number;
35   gross_margin_pct: number;
36   ebitda_margin_pct: number;
37   net_margin_pct: number;
38 }
39
40 export interface SimulationResult {
41   original: CascadaResult;
42   simulated: CascadaResult;
43   delta_ebitda: number;
44   delta_margin: number;
45 }
46
47 export interface PricingResult {
48   costo_materiales: number;
49   costo_mano_obra: number;
50   costo_fijo_unitario: number;
51   costo_total_unitario: number;
52   precio_sin_itbms: number;
53   itbms: number;
54   precio_final: number;
55   ganancia_por_unidad: number;
56 }
```

```
57
58 /** Categoria de unidad de medida */
59 export type UnitCategory = "weight" | "volume";
60
61 /** Definicion de una unidad de medida (built-in o custom) */
62 export interface UnitOfMeasure {
63   id: string;
64   label: string;
65   category: UnitCategory;
66   /** Cuantos gramos (peso) o ml (volumen) equivale 1 unidad */
67   conversionFactor: number;
68   isCustom?: boolean;
69 }
70
71 export interface Ingredient {
72   id: string;
73   nombre: string;
74   /** Monto total pagado por la compra (ej: $100 por 10 libras) */
75   costoAdquisicion: number;
76   /** Cantidad comprada (ej: 10 libras). Si no se indica, asume 1 */
77   cantidadCompra?: number;
78   /** ID de la unidad de compra (ej: "quintal", "kg", "lb", "galon", "litro") */
79   unidadCompraId: string;
80   /** Cantidad utilizada por unidad de producto */
```

... 239 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
320       let m = refMonth - 5;
321       let y = refYear;
322       if (m < 1) { m += 12; y -= 1; }
323       return { from: { year: y, month: m }, to };
324     }
325     case "last_year": {
326       let m = refMonth;
327       let y = refYear - 1;
328       return { from: { year: y, month: m + 1 > 12 ? 1 : m + 1 }, to };
329     }
330     case "ytd":
331       return { from: { year: refYear, month: 1 }, to };
332   }
333 }
334
335 // ============================================
336 // OXIGENO (Ratios de liquidez)
337 // ============================================
338
339 export function computeOxigeno(r: FinancialRecord) {
340   const diasCalle =
341     r.revenue > 0 ? (r.accounts_receivable / r.revenue) * 30 : 0;
342   const diasInventario =
343     r.cogs > 0 ? (r.inventory / r.cogs) * 30 : 0;
344   const diasProveedor =
345     r.cogs > 0 ? (r.accounts_payable / r.cogs) * 30 : 0;
346   const ccc = diasCalle + diasInventario - diasProveedor;
347
348   const totalLiquido = r.cash_balance + r.accounts_receivable;
349   const pasivoCorto = r.accounts_payable + r.bank_debt;
350   const pruebaAcida = pasivoCorto > 0 ? totalLiquido / pasivoCorto : 0;
351
352   const dineroAtrapado = r.accounts_receivable + r.inventory;
353
```

```
  const cascada = computeCascada(r);
  const coberturaBancaria =
    r.interest_expense > 0 ? cascada.ebitda / r.interest_expense : 10;

  return {
    diasCalle,
    diasInventario,
    diasProveedor,
    ccc,
    pruebaAcida,
    dineroAtrapado,
    coberturaBancaria,
    totalLiquido,
    pasivoCorto,
  };
}
```


Página 9 de 33

frontend/src/lib/alerts.ts

Total: 557 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  /**
2   * Motor de Alertas Centralizado — Mi Director Financiero PTY
3   * Genera alertas contextuales basadas en datos financieros,
4   * legales y laborales del negocio.
5   */
6
7  import type { FinancialRecord } from "@/lib/calculations";
8  import { computeCascada, computeOxigeno } from "@/lib/calculations";
9
10 // ============================================
11 // TIPOS
12 // ============================================
13
14 export type AlertPriority = "red" | "orange" | "yellow" | "green";
15 export type AlertCategory = "dgi" | "capital_humano" | "liquidez" | "rentabilidad" | "legal"
| "inventario" | "facturacion";
16
17 export interface StrategicAlert {
18   id: string;
19   priority: AlertPriority;
20   category: AlertCategory;
21   title: string;
22   message: string;
23   emoji: string;
24   /** Texto corto para el Smart Prompt placeholder */
25   promptHint: string;
26 }
27
28 // ============================================
29 // MOTOR DE ALERTAS
30 // ============================================
31
32 export function computeAlerts(record: FinancialRecord): StrategicAlert[] {
33   const alerts: StrategicAlert[] = [];
34   const cascada = computeCascada(record);
35   const ox = computeOxigeno(record);
36
37   // ---- RENTABILIDAD ----
38
39   // Utilidad Neta negativa
40   if (cascada.net_income < 0) {
41     alerts.push({
42       id: "net_loss",
43       priority: "red",
44       category: "rentabilidad",
45       title: "Perdida Neta",
46       message: `Tu negocio esta perdiendo $${Math.abs(cascada.net_income).toLocaleString("es-
PA")} al mes. Revisa costos y gastos urgentemente.`,
47       emoji: "🚨",
48       promptHint: "ALERTA: Tu negocio esta en numeros rojos este mes",
49     });
50   }
51
52   // EBITDA negativo
53   if (cascada.ebitda < 0) {
54     alerts.push({
```

```
55        id: "ebitda_negative",
56        priority: "red",
57        category: "rentabilidad",
58        title: "EBITDA Negativo",
59        message: `Tu operacion no genera caja. EBITDA:
-$${Math.abs(cascada.ebitda).toLocaleString("es-PA")}. Los gastos operativos superan tu utilidad
bruta.`,
60        emoji: "💀",
61        promptHint: "CRITICO: Tu operacion no genera caja suficiente",
62      });
63    } else if (cascada.ebitda_margin_pct < 10) {
64      alerts.push({
65        id: "ebitda_low",
66        priority: "orange",
67        category: "rentabilidad",
68        title: "Margen EBITDA bajo",
69        message: `Tu margen EBITDA es ${cascada.ebitda_margin_pct.toFixed(1)}% (meta: >15%).
Necesitas optimizar costos o subir precios.`,
70        emoji: "📉",
71        promptHint: `Tu margen EBITDA es solo ${cascada.ebitda_margin_pct.toFixed(1)}%`,
72      });
73    }
74
75    // Margen Bruto bajo
76    if (cascada.gross_margin_pct < 25 && record.revenue > 0) {
77      alerts.push({
78        id: "gross_margin_low",
79        priority: "orange",
80        category: "rentabilidad",
```

... 427 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
508   return alerts;
509 }
510
511 // ============================================
512 // FACTURACION ELECTRONICA ALERTS
513 // ============================================
514
515 /**
516  * Genera alertas de facturacion basadas en el segmento DGI.
517  * Usa detectarSegmentoLocal() del modulo ventas para determinar
518  * si la sociedad debe migrar a facturacion electronica (PAC).
519  */
520 export function computeFacturacionAlerts(societyId: string): StrategicAlert[] {
521   // Importacion dinamica para evitar dependencia circular
522   // En runtime, ventas-segmento.ts no depende de alerts.ts
523   try {
524     // eslint-disable-next-line @typescript-eslint/no-require-imports
525     const { detectarSegmentoLocal } = require("@/lib/ventas-segmento");
526     const resultado = detectarSegmentoLocal(societyId);
527
528     const alerts: StrategicAlert[] = [];
529
530     if (resultado.alertaNivel === "rojo") {
531       alerts.push({
532         id: "facturacion-pac-obligatorio",
533         priority: "red",
534         category: "facturacion",
535         title: "Migracion a PAC Obligatoria",
536         message: resultado.mensajeAlerta || `Has superado los limites DGI (B/.36,000/ano o
```


Página 11 de 33

```
100 facturas/mes). Debes activar facturacion electronica con un PAC certificado.`,
537         emoji: "📄",
538         promptHint: "URGENTE: Debes activar facturacion electronica PAC",
539       });
540     } else if (resultado.alertaNivel === "amarillo") {
541       alerts.push({
542         id: "facturacion-pac-preventivo",
543         priority: "yellow",
544         category: "facturacion",
545         title: "Acercandote al Limite de Facturacion",
546         message: resultado.mensajeAlerta || `Estas al ${Math.max(resultado.pctIngresos,
resultado.pctFacturas).toFixed(0)}% del limite DGI. Considera preparar la migracion a facturacion
electronica.`,
547         emoji: "📋",
548         promptHint: "Estas cerca del limite para facturacion electronica",
549       });
550     }
551
552     return alerts;
553   } catch {
554     // Si ventas-segmento no esta disponible, no generar alertas
555     return [];
556   }
557 }
```


Página 12 de 33

frontend/src/components/rrhh/MiRRHH.tsx

Total: 2224 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  "use client";
2  import React, { useState, useEffect, useMemo, useCallback } from "react";
3  import {
4    Users,
5    UserPlus,
6    Trash2,
7    Edit3,
8    Save,
9    X,
10   Search,
11   DollarSign,
12   FileText,
13   ChevronDown,
14   ChevronUp,
15   Lock,
16   Copy,
17   Briefcase,
18   User,
19   ClipboardList,
20   Check,
21   Clock,
22   Gift,
23   CreditCard,
24   ScrollText,
25   CalendarDays,
26   Calculator,
27 } from "lucide-react";
28 import HorasExtrasTab from "./HorasExtrasTab";
29 import BonificacionesTab from "./BonificacionesTab";
30 import PrestamosTab from "./PrestamosTab";
31 import ContratosTab from "./ContratosTab";
32 import AsistenciaTab from "./AsistenciaTab";
33 import SimuladorNomina from "@/components/rrhh/SimuladorNomina";
34 import {
35   type PersonalRecord,
36   type TipoPersonal,
37   type TipoDocumento,
38   type TipoContrato,
39   type FormaPago,
40   type Jornada,
41   type EstadoPersonal,
42   type ModalidadPagoFreelance,
43   type PlanillaMensualEntry,
44   type PlanillaMensual,
45   type PagoFreelancerEntry,
46   type PagosFreelancerMes,
47   DEFAULT_TASAS,
48   FREQUENCY_CONFIGS,
49   calcularISRMensual,
50   calcularISRGastosRepresentacion,
51   loadPersonal,
52   savePersonal,
53   loadPlanillas,
54   savePlanillas,
55   loadPagosFreelancers,
56   savePagosFreelancers,
```

```
  genId,
} from "@/lib/rrhh-types";

// ============================================
// CONSTANTS
// ============================================

const MONTHS = [
  "Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",
  "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre",
];

const TIPO_PERSONAL_LABELS: Record<TipoPersonal, string> = {
  empleado: "Empleado",
  freelancer: "Freelancer",
};

const TIPO_DOCUMENTO_LABELS: Record<TipoDocumento, string> = {
  cedula: "Cedula",
  pasaporte: "Pasaporte",
  carnet_residente: "Carnet de Residente",
};

const TIPO_CONTRATO_LABELS: Record<TipoContrato, string> = {
```

... 2094 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
            )}
            {isClosed && (
              <div className="flex items-center gap-2 text-sm text-amber-700 bg-amber-50 px-4 py-2 rounded-lg">
                <Lock className="w-4 h-4" />
                Estos pagos estan cerrados y no se pueden editar.
              </div>
            )}
          </div>

          {/* Summary Cards */}
          <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
            <SummaryCard label="Total Bruto" value={fmtMoney(totals.monto_bruto)} color="blue" />
            <SummaryCard label="Total ITBMS" value={fmtMoney(totals.itbms)} color="slate" />
            <SummaryCard label="Total Ret. ISR" value={fmtMoney(totals.retencion_isr)} color="red" />
            <SummaryCard label="Total Neto" value={fmtMoney(totals.monto_neto)} color="green" />
          </div>
        </>
      )}
    </div>
  );
}

// ============================================
// SHARED: SUMMARY CARD
// ============================================

function SummaryCard({
  label,
  value,
  color,
```

```
}: {
  label: string;
  value: string;
  color: "blue" | "red" | "green" | "amber" | "slate";
}) {
  const colorMap = {
    blue: "bg-blue-50 border-blue-200 text-blue-800",
    red: "bg-red-50 border-red-200 text-red-800",
    green: "bg-green-50 border-green-200 text-green-800",
    amber: "bg-amber-50 border-amber-200 text-amber-800",
    slate: "bg-slate-50 border-slate-200 text-slate-800",
  };

  return (
    <div className={`rounded-lg border p-3 ${colorMap[color]}`}>
      <div className="text-[11px] font-medium opacity-70 mb-0.5">{label}</div>
      <div className="text-sm font-semibold">{value}</div>
    </div>
  );
}
```


Página 15 de 33

frontend/src/lib/ventas-storage.ts

Total: 367 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  /**
2   * Libro de Ventas — Storage Layer
3   * Mi Director Financiero PTY
4   * CRUD localStorage para ventas (facturacion)
5   *
6   * Clave localStorage: midf_ventas
7   * Tasa ITBMS: 7%
8   */
9
10 import type {
11   Venta,
12   VentaInput,
13   VentasFilter,
14   ResumenMensual,
15   OrigenVenta,
16   MetodoPagoVenta,
17 } from "./ventas-types";
18
19 const VENTAS_KEY = "midf_ventas";
20 const ITBMS_RATE = 0.07;
21
22 // ============================================
23 // HELPERS
24 // ============================================
25
26 function generateId(): string {
27   if (typeof crypto !== "undefined" && crypto.randomUUID) {
28     return crypto.randomUUID();
29   }
30   return `v-${Date.now()}-${Math.random().toString(36).slice(2, 9)}`;
31 }
32
33 /** Lee todas las ventas desde localStorage */
34 export function readVentas(): Venta[] {
35   if (typeof window === "undefined") return [];
36   try {
37     const raw = localStorage.getItem(VENTAS_KEY);
38     return raw ? JSON.parse(raw) : [];
39   } catch {
40     return [];
41   }
42 }
43
44 /** Escribe todas las ventas a localStorage */
45 export function writeVentas(ventas: Venta[]): void {
46   if (typeof window === "undefined") return;
47   try {
48     localStorage.setItem(VENTAS_KEY, JSON.stringify(ventas));
49   } catch {
50     // localStorage lleno o no disponible
51   }
52 }
53
54 /** Fecha de hoy en formato ISO "YYYY-MM-DD" */
55 function todayISO(): string {
56   const d = new Date();
```

```
57   const y = d.getFullYear();
58   const m = String(d.getMonth() + 1).padStart(2, "0");
59   const day = String(d.getDate()).padStart(2, "0");
60   return `${y}-${m}-${day}`;
61 }
62
63 /** Hora actual en formato "HH:MM" */
64 function nowTime(): string {
65   const d = new Date();
66   const h = String(d.getHours()).padStart(2, "0");
67   const min = String(d.getMinutes()).padStart(2, "0");
68   return `${h}:${min}`;
69 }
70
71 // ============================================
72 // CONSULTAS
73 // ============================================
74
75 /**
76  * Obtiene ventas filtradas por societyId y filtros opcionales.
77  * Retorna ordenadas por fecha descendente (mas reciente primero).
78  */
79 export function getVentas(societyId: string, filters?: VentasFilter): Venta[] {
80   let ventas = readVentas().filter((v) => v.societyId === societyId);
```

... 237 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
318
319 /**
320  * Calcula el resumen mensual de ventas (no anuladas) para un mes especifico.
321  * @param year - Anio (e.g. 2026)
322  * @param month - Mes 1-12
323  */
324 export function getVentasResumenMensual(
325   societyId: string,
326   year: number,
327   month: number
328 ): ResumenMensual {
329   // Construir rango de fechas del mes
330   const mesStr = String(month).padStart(2, "0");
331   const desde = `${year}-${mesStr}-01`;
332   // Ultimo dia del mes
333   const lastDay = new Date(year, month, 0).getDate();
334   const hasta = `${year}-${mesStr}-${String(lastDay).padStart(2, "0")}`;
335
336   const ventas = getVentas(societyId, { desde, hasta, anulada: false });
337
338   const resumen: ResumenMensual = {
339     total: 0,
340     totalItbms: 0,
341     count: ventas.length,
342     byOrigen: emptyByOrigen(),
343     byMetodoPago: emptyByMetodoPago(),
344   };
345
346   for (const v of ventas) {
347     resumen.total += v.montoTotal;
348     resumen.totalItbms += v.itbms;
349
350     // Acumular por origen
351     resumen.byOrigen[v.origen].total += v.montoTotal;
```


Página 17 de 33

```
    resumen.byOrigen[v.origen].count++;

    // Acumular por metodo de pago
    resumen.byMetodoPago[v.metodoPago].total += v.montoTotal;
    resumen.byMetodoPago[v.metodoPago].count++;
  }

  // Redondear totales a 2 decimales
  resumen.total = Math.round(resumen.total * 100) / 100;
  resumen.totalItbms = Math.round(resumen.totalItbms * 100) / 100;

  return resumen;
}

// Exportar constante de tasa ITBMS para reutilizar
export { ITBMS_RATE, VENTAS_KEY };
```


Página 18 de 33

frontend/src/lib/analytics/cb-insights-detector.ts

Total: 628 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  /**
2   * DETECTOR DE LAS 6 CAUSAS CB INSIGHTS
3   * Evalua la salud del negocio basado en datos financieros locales.
4   * Fuente: CB Insights "Top 20 Reasons Startups Fail" (2023)
5   *
6   * Usa computeCascada() y datos del FinancialRecord existente,
7   * NO requiere Supabase.
8   */
9
10 import type { FinancialRecord } from "@/lib/calculations";
11 import { computeCascada } from "@/lib/calculations";
12
13 // ============================================
14 // TIPOS
15 // ============================================
16
17 export type NivelRiesgo = "critico" | "precaucion" | "en_orden" | "sin_datos";
18
19 export interface ResultadoCausa {
20   id: number;
21   titulo: string;
22   subtitulo: string;
23   nivel: NivelRiesgo;
24   valor_actual: string;
25   umbral: string;
26   tendencia: "mejorando" | "estable" | "empeorando" | "sin_datos";
27   mensaje: string;
28   accion: string;
29   emoji: string;
30   base_cb: string;
31 }
32
33 export interface ResultadoCBInsights {
34   calculado_en: string;
35   causas: ResultadoCausa[];
36   score_salud: number; // 0-100
37   nivel_general: NivelRiesgo;
38   resumen: string;
39 }
40
41 // ============================================
42 // CONSTANTES
43 // ============================================
44
45 /** Factor costo real empresa Panama (CSS 12.25% + RP 1.50% + SE 1.50% = 15.25%, redondeado
1.36x) */
46 const FACTOR_COSTO_EMPRESA = 1.36;
47
48 // Umbrales
49 const RUNWAY_CRITICO_MESES = 3;
50 const RUNWAY_PRECAUCION_MESES = 6;
51 const PAYROLL_CRITICO_PCT = 40;
52 const PAYROLL_PRECAUCION_PCT = 30;
53 const GROSS_MARGIN_CRITICO_PCT = 15;
54 const GROSS_MARGIN_PRECAUCION_PCT = 20;
55 const COGS_CRITICO_PCT = 70;
```


Página 19 de 33

```
56 const COGS_PRECAUCION_PCT = 60;
57 const GROWTH_CRITICO_PCT = 0;
58 const GROWTH_PRECAUCION_PCT = 5;
59
60 // ============================================
61 // HELPERS
62 // ============================================
63
64 function formatPct(n: number): string {
65   return `${n.toFixed(1)}%`;
66 }
67
68 function formatMeses(n: number): string {
69   if (n >= 99) return ">99 meses";
70   return `${n.toFixed(1)} meses`;
71 }
72
73 function formatBalboas(n: number): string {
74   return `B/.${Math.abs(n).toLocaleString("es-PA", { minimumFractionDigits: 0,
maximumFractionDigits: 0 })}`;
75 }
76
77 /**
78  * Calcula tendencia simple comparando 2-3 valores mensuales.
79  * Retorna 'mejorando' si el ultimo es mayor, 'empeorando' si es menor.
80  */
```

... 498 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
579   }
580
581   if (criticos.length > 0) {
582     const nombres = criticos.map((c) => c.titulo).join(", ");
583     return `Atencion urgente en: ${nombres}. Score de salud: ${score}/100.`;
584   }
585
586   if (precauciones.length > 0) {
587     return `Tu negocio esta estable pero con ${precauciones.length} areas de precaucion.
Score: ${score}/100.`;
588   }
589
590   return `Tu negocio esta saludable en las 6 dimensiones CB Insights. Score: ${score}/100.`;
591 }
592
593 // ============================================
594 // FUNCION PRINCIPAL
595 // ============================================
596
597 /**
598  * Ejecuta el analisis completo de las 6 causas CB Insights.
599  *
600  * @param record - FinancialRecord del periodo actual (de localStorage)
601  * @param ventasMensuales - Ventas mensuales para calcular tendencia (opcional)
602  * @returns ResultadoCBInsights con las 6 causas evaluadas y score general
603  */
604 export function detectarCausasCBInsights(
605   record: FinancialRecord | null,
606   ventasMensuales?: { mes: string; total: number }[]
607 ): ResultadoCBInsights {
608   const causas: ResultadoCausa[] = [
609     detectarSinMercado(record, ventasMensuales),
```

```
    detectarSinCaja(record),
    detectarEquipoIncorrecto(record),
    detectarCompetencia(record),
    detectarProblemaPrecios(record),
    detectarModeloNoEscalable(record),
  ];

  const score_salud = calcularScoreSalud(causas);
  const nivel_general = determinarNivelGeneral(causas);
  const resumen = generarResumen(causas, score_salud, nivel_general);

  return {
    calculado_en: new Date().toISOString(),
    causas,
    score_salud,
    nivel_general,
    resumen,
  };
}
```


Página 21 de 33

frontend/src/lib/rrhh/nomina-calculator.ts

Total: 375 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  /**
2   * Motor de Calculo de Nomina — Mi Director Financiero PTY
3   * Base legal: Ley 462 de 2025 (CSS/SE) · Codigo de Trabajo · ISR Art. 700
4   *
5   * TASAS CORREGIDAS (Bug #1 — 2026-03-04):
6   *   CSS Patronal = 12.25% (NO 13.25%)
7   *   Total cargas patronales = 12.25 + 1.50 + 1.50 = 15.25%
8   *
9   * ISR (Bug #2 — 2026-03-04):
10  *   1. Deducir 11% (CSS 9.75% + SE 1.25%) del bruto
11  *   2. Anualizar x12
12  *   3. Tabla DGI: 0-11K -> 0%, 11K-50K -> 15%, >50K -> 25%
13  *   4. Dividir /12
14  *   Verificacion: B/.1,500 bruto -> ISR mensual ~B/.62.75
15  */
16
17 import type {
18   EmpleadoInput,
19   DesgloseCostosEmpleador,
20   DesglosePagoEmpleado,
21   ResultadoNominaEmpleado,
22   ResultadoNominaTotal,
23 } from "./nomina-types";
24
25 // ============================================
26 // TASAS LEGALES (Ley 462/2025 corregida)
27 // ============================================
28
29 export const TASAS_NOMINA = {
30   // Cargas patronales
31   CSS_PATRONAL: 0.1225,              // 12.25% (CORREGIDO - Ley 462/2025)
32   SE_PATRONAL: 0.0150,               // 1.50%
33   RIESGOS_PROFESIONALES: 0.0150,     // 1.50%
34   // Prestaciones sociales
35   DECIMOTERCER_MES: 1 / 12,          // 8.33%
36   VACACIONES: 1 / 24,                // 4.17%
37   CESANTIA: 0.0225,                  // 2.25%
38   // Retenciones al empleado
39   CSS_EMPLEADO: 0.0975,              // 9.75%
40   SE_EMPLEADO: 0.0125,               // 1.25%
41   SIACAP_MENSUAL: 10.00,             // B/.10/mes (solo sector publico)
42   // ISR
43   ISR_MINIMO_NO_IMPONIBLE: 11_000,
44   ISR_TRAMO_1_LIMITE: 50_000,
45   ISR_TRAMO_1_TASA: 0.15,
46   ISR_TRAMO_2_TASA: 0.25,
47 } as const;
48
49 // ============================================
50 // UTILITY
51 // ============================================
52
53 function r2(n: number): number {
54   return Math.round(n * 100) / 100;
55 }
56
```

```
57 // ============================================
58 // 1. COSTOS DEL EMPLEADOR
59 // ============================================
60
61 export function calcularCostosEmpleador(salario_bruto: number): DesgloseCostosEmpleador {
62   if (salario_bruto <= 0) {
63     return {
64       salario_bruto: 0,
65       css_patronal_pct: TASAS_NOMINA.CSS_PATRONAL * 100,
66       css_patronal_monto: 0,
67       se_patronal_pct: TASAS_NOMINA.SE_PATRONAL * 100,
68       se_patronal_monto: 0,
69       riesgos_profesionales_pct: TASAS_NOMINA.RIESGOS_PROFESIONALES * 100,
70       riesgos_profesionales_monto: 0,
71       decimotercer_mes_pct: r2(TASAS_NOMINA.DECIMOTERCER_MES * 100),
72       decimotercer_mes_monto: 0,
73       vacaciones_pct: r2(TASAS_NOMINA.VACACIONES * 100),
74       vacaciones_monto: 0,
75       cesantia_pct: TASAS_NOMINA.CESANTIA * 100,
76       cesantia_monto: 0,
77       total_cargas_porcentuales: 0,
78       total_prestaciones: 0,
79       total_carga_adicional: 0,
80       costo_total_empleador: 0,
```

... 245 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
326     empleador.cesantia_monto
327   );
328   if (Math.abs(empleador.total_carga_adicional - cargasEsperadas) > 0.02) {
329     errores.push(
330       `Cargas adicionales (${empleador.total_carga_adicional}) no coinciden con suma
individual (${cargasEsperadas})`
331     );
332   }
333
334   // Validate CSS Patronal rate is 12.25% NOT 13.25%
335   if (empleador.css_patronal_pct !== 12.25) {
336     errores.push(
337       `CSS Patronal debe ser 12.25% (Ley 462/2025), se encontro
${empleador.css_patronal_pct}%`
338     );
339   }
340
341   // Validate employee retenciones add up
342   const retencionesEsperadas = r2(
343     empleado_recibe.css_empleado_monto +
344     empleado_recibe.se_empleado_monto +
345     empleado_recibe.siacap_mensual +
346     empleado_recibe.isr_mensual
347   );
348   if (Math.abs(empleado_recibe.total_retenciones - retencionesEsperadas) > 0.02) {
349     errores.push(
350       `Total retenciones (${empleado_recibe.total_retenciones}) no coincide con suma
individual (${retencionesEsperadas})`
351     );
352   }
353
354   // Validate factor multiplicador is reasonable (should be ~1.30-1.40 typically)
355   if (empleador.factor_multiplicador > 0 && (empleador.factor_multiplicador < 1.1 ||
empleador.factor_multiplicador > 2.0)) {
```


Página 23 de 33

```
    errores.push(
      `Factor multiplicador fuera de rango razonable: ${empleador.factor_multiplicador}x`
    );
  }

  // Validate net salary is positive
  if (empleado_recibe.salario_neto < 0) {
    errores.push(`Salario neto negativo: B/.${empleado_recibe.salario_neto}`);
  }

  // Cross-validate resumen
  if (Math.abs(resumen.lo_que_paga_empresa - empleador.costo_total_empleador) > 0.02) {
    errores.push("Resumen: lo_que_paga_empresa no coincide con costo_total_empleador");
  }
  if (Math.abs(resumen.lo_que_recibe_empleado - empleado_recibe.salario_neto) > 0.02) {
    errores.push("Resumen: lo_que_recibe_empleado no coincide con salario_neto");
  }

  return errores;
}
```


Página 24 de 33

frontend/src/lib/analytics/forecast-engine.ts

Total: 508 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  /**
2   * Motor de Forecasting 12 Meses — MDF PTY
3   * Metodo: Regresion lineal (minimos cuadrados) + supuestos de costos historicos
4   * Fuente de datos: localStorage (midf_ventas + FinancialRecord)
5   *
6   * GAP-4: Sistema de Proyecciones Financieras
7   */
8
9  import type { FinancialRecord } from "@/lib/calculations";
10 import { readVentas } from "@/lib/ventas-storage";
11
12 // ============================================
13 // TIPOS
14 // ============================================
15
16 export interface PuntoHistorico {
17   mes: number;   // 1-12
18   anio: number;
19   revenue: number;
20 }
21
22 export interface SupuestosForecast {
23   cogs_pct: number;          // % COGS sobre Revenue
24   opex_pct: number;          // % OPEX sobre Revenue
25   nomina_pct: number;        // % Nomina sobre Revenue
26   tasa_crecimiento: number;  // % crecimiento mensual (for manual/promedio mode)
27 }
28
29 export interface MesProyectado {
30   anio: number;
31   mes: number;
32   mesLabel: string;          // "Ene 2026"
33   revenue: number;
34   cogs: number;
35   opex: number;
36   nomina: number;
37   utilidad_bruta: number;
38   ebitda: number;
39   utilidad_neta: number;
40   es_real: boolean;
41   es_editado: boolean;
42 }
43
44 export interface ResultadoForecast {
45   historico: MesProyectado[];
46   proyeccion: MesProyectado[];
47   supuestos: SupuestosForecast;
48   metodo: "regresion_lineal" | "promedio_simple" | "sin_datos";
49   kpis: {
50     revenue_total_12m: number;
51     utilidad_neta_12m: number;
52     margen_neto_promedio: number;
53     mes_breakeven: number | null;   // Indice del mes donde UN pasa de negativo a positivo
54     tendencia: "creciente" | "estable" | "decreciente";
55   };
56 }
```

```
57
58 // ============================================
59 // CONSTANTES
60 // ============================================
61
62 const MESES = [
63   "Ene", "Feb", "Mar", "Abr", "May", "Jun",
64   "Jul", "Ago", "Sep", "Oct", "Nov", "Dic",
65 ];
66
67 /** ISR estimado para Panama (25% sobre EBITDA positivo) */
68 const ISR_RATE = 0.25;
69
70 /** Supuestos por defecto cuando no hay datos */
71 const DEFAULT_SUPUESTOS: SupuestosForecast = {
72   cogs_pct: 40,
73   opex_pct: 20,
74   nomina_pct: 15,
75   tasa_crecimiento: 5,
76 };
77
78 // localStorage key para ediciones manuales de revenue
79 const FORECAST_EDITS_KEY = "midf_forecast_edits";
80
```

... 378 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
459     if (a.anio !== b.anio) return a.anio - b.anio;
460     return a.mes - b.mes;
461   });
462
463   return puntos;
464 }
465
466 // ============================================
467 // HELPER: Calcular supuestos desde FinancialRecord
468 // ============================================
469
470 /**
471  * Deriva supuestos (%) del FinancialRecord guardado en localStorage.
472  * Si no hay record o revenue es cero, retorna valores por defecto.
473  */
474 export function calcularSupuestosDesdeRecord(
475   record: FinancialRecord | null
476 ): SupuestosForecast {
477   if (!record || record.revenue <= 0) {
478     return { ...DEFAULT_SUPUESTOS };
479   }
480
481   const rev = record.revenue;
482
483   const cogs_pct = round2((record.cogs / rev) * 100);
484   const opex_pct = round2(
485     ((record.opex_rent + record.opex_other) / rev) * 100
486   );
487   const nomina_pct = round2((record.opex_payroll / rev) * 100);
488
489   // Tasa de crecimiento: default 5% si no hay datos multi-periodo
490   const tasa_crecimiento = 5;
491
492   return {
```


Página 26 de 33

```
    cogs_pct: clamp(cogs_pct, 0, 95),
    opex_pct: clamp(opex_pct, 0, 80),
    nomina_pct: clamp(nomina_pct, 0, 60),
    tasa_crecimiento,
  };
}

function clamp(value: number, min: number, max: number): number {
  return Math.min(max, Math.max(min, value));
}

// ==========================================
// EXPORTS ADICIONALES
// ==========================================

export { MESES, DEFAULT_SUPUESTOS };
```


Página 27 de 33

frontend/src/services/facturacion/FacturacionService.ts

Total: 181 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  /**
2   * FacturacionService — Unico punto de entrada al sistema de facturacion.
3   *
4   * El resto de la app (incluyendo F2V10DraftService) solo habla con
5   * FacturacionService, nunca directamente con el adaptador PAC.
6   *
7   * Hoy: lee de localStorage (ManualEntryAdapter).
8   * Futuro PAC: lee del servidor del PAC. Nada cambia aqui.
9   */
10
11 import { PACFactory } from "./pac/PACFactory";
12 import type { PACProvider, PeriodSummary } from "./pac/PACProvider.interface";
13
14 // ============================================================
15 // Tipos de resultado para el F2 V10
16 // ============================================================
17
18 export interface ResumenFacturacion {
19   total_facturado_neto: number;
20   total_itbms_cobrado: number;
21   por_mes: PeriodSummary["by_month"];
22   fuente: "MANUAL" | "PAC_SYNC";
23   documentos_count: number;
24   discrepancia_posible: boolean;
25   advertencia?: string;
26 }
27
28 export interface IntegrationStatus {
29   mode: "MANUAL" | "PAC_ACTIVE";
30   pac_name: string;
31   pac_environment: string;
32   connected: boolean;
33   latency_ms: number;
34 }
35
36 export interface FreeInvoicerLimits {
37   docs_this_month: number;
38   docs_monthly_limit: number;
39   docs_pct: number;
40   docs_alert: boolean;
41   revenue_annual: number;
42   revenue_annual_limit: number;
43   revenue_pct: number;
44   revenue_alert: boolean;
45   requires_pac_now: boolean;
46   alert_message?: string;
47 }
48
49 // ============================================================
50 // Limites DGI — Resolucion 201-6299
51 // ============================================================
52
53 const FREE_INVOICER_DOCS_LIMIT = 100;
54 const FREE_INVOICER_REVENUE_LIMIT = 36_000;
55 const DOCS_ALERT_PCT = 0.80;
56 const REVENUE_ALERT_PCT = 0.85;
```

```
57
58 // ============================================================
59 // Service
60 // ============================================================
61
62 export class FacturacionService {
63   private pac: PACProvider;
64
65   constructor() {
66     this.pac = PACFactory.createFromEnv();
67   }
68
69   // ── Para el F2 V10 ──────────────────────────────────────
70
71   async getResumenParaDeclaracion(params: {
72     societyId: string;
73     periodoFiscal: number;
74     ruc?: string;
75   }): Promise<ResumenFacturacion> {
76     const summary = await this.pac.getPeriodSummary({
77       ruc: params.ruc || params.societyId,
78       fiscal_year: params.periodoFiscal,
79     });
80
```

... 51 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
132
133     const docs = monthSummary.document_count;
134     const revenue = yearSummary.net_total;
135
136     const docs_pct = (docs / FREE_INVOICER_DOCS_LIMIT) * 100;
137     const revenue_pct = (revenue / FREE_INVOICER_REVENUE_LIMIT) * 100;
138     const requires_pac_now =
139       docs >= FREE_INVOICER_DOCS_LIMIT ||
140       revenue >= FREE_INVOICER_REVENUE_LIMIT;
141
142     return {
143       docs_this_month: docs,
144       docs_monthly_limit: FREE_INVOICER_DOCS_LIMIT,
145       docs_pct,
146       docs_alert: docs_pct >= DOCS_ALERT_PCT * 100,
147       revenue_annual: revenue,
148       revenue_annual_limit: FREE_INVOICER_REVENUE_LIMIT,
149       revenue_pct,
150       revenue_alert: revenue_pct >= REVENUE_ALERT_PCT * 100,
151       requires_pac_now,
152       alert_message: requires_pac_now
153         ? "Has superado los limites del facturador gratuito DGI. " +
154           "Necesitas contratar un PAC certificado para seguir facturando."
155         : undefined,
156     };
157   }
158
159   // ── Sincronizacion futura ──────────────────────────────
160
161   async syncWithPAC(_params: {
162     societyId: string;
163     from: Date;
164     to: Date;
165   }): Promise<void> {
```

```
    if (this.pac.name === "MANUAL_ENTRY") {
      throw new Error(
        "La sincronizacion con PAC no esta disponible en modo manual. " +
          "Activa un PAC certificado DGI en Configuracion > Facturacion."
      );
    }
    // Logica de sync cuando exista un adaptador PAC real
  }
}

// Singleton para uso en componentes
let _instance: FacturacionService | null = null;
export function getFacturacionService(): FacturacionService {
  if (!_instance) _instance = new FacturacionService();
  return _instance;
}
```


Página 30 de 33

frontend/src/lib/reportes/reporte-engine.ts

Total: 333 líneas | Mostrando primeras 80 y últimas 50 líneas

PRIMERAS 80 LÍNEAS

```
1  /**
2   * Motor de Reportes Mensuales — MDF PTY
3   * Genera un snapshot inmutable del estado financiero del negocio
4   * en un periodo especifico. Usa datos de localStorage.
5   */
6
7  import type { FinancialRecord, CascadaResult } from "@/lib/calculations";
8  import { computeCascada } from "@/lib/calculations";
9  import { computeAlerts, type StrategicAlert } from "@/lib/alerts";
10 import { getVentas } from "@/lib/ventas-storage";
11 import { loadPersonal } from "@/lib/rrhh-types";
12 import type { Venta } from "@/lib/ventas-types";
13 import { METODO_PAGO_LABELS, ORIGEN_VENTA_LABELS } from "@/lib/ventas-types";
14
15 // ============================================
16 // TIPOS
17 // ============================================
18
19 export interface ReporteMensual {
20   id: string;
21   societyId: string;
22   periodo: { anio: number; mes: number };
23   periodoLabel: string;       // "Marzo 2026"
24   estado: "borrador" | "cerrado";
25   cerrado_en: string | null;
26   created_at: string;
27
28   // Secciones del reporte
29   resumen_ejecutivo: {
30     ingresos: number;
31     costos: number;
32     utilidad_bruta: number;
33     gastos_operativos: number;
34     ebitda: number;
35     utilidad_neta: number;
36     margen_neto_pct: number;
37     total_empleados: number;
38     total_facturas: number;
39   };
40
41   cascada: CascadaResult | null;
42
43   ventas_resumen: {
44     total_ventas: number;
45     cantidad_facturas: number;
46     por_metodo_pago: { metodo: string; total: number; cantidad: number }[];
47     por_origen: { origen: string; total: number; cantidad: number }[];
48   } | null;
49
50   alertas_activas: {
51     total: number;
52     criticas: number;
53     precaucion: number;
54     lista: { titulo: string; nivel: string; mensaje: string }[];
55   } | null;
56 }
```


Página 31 de 33

```
57
58 // ============================================
59 // CONSTANTES
60 // ============================================
61
62 const MESES = [
63   "Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",
64   "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre",
65 ];
66
67 const STORAGE_KEY = "midf_reportes_cierres";
68
69 // ============================================
70 // HELPERS
71 // ============================================
72
73 function generateId(): string {
74   if (typeof crypto !== "undefined" && crypto.randomUUID) {
75     return crypto.randomUUID();
76   }
77   return `rpt-${Date.now()}-${Math.random().toString(36).slice(2, 9)}`;
78 }
79
80 /** Lee reportes guardados desde localStorage */
```

... 203 líneas omitidas ...

ÚLTIMAS 50 LÍNEAS

```
284     .filter((r) => r.societyId === societyId)
285     .sort((a, b) => {
286       if (a.periodo.anio !== b.periodo.anio)
287         return b.periodo.anio - a.periodo.anio;
288       return b.periodo.mes - a.periodo.mes;
289     });
290 }
291
292 /**
293  * Cierra un reporte (accion irreversible).
294  * Una vez cerrado, el reporte queda como snapshot inmutable
295  * y no se puede modificar ni eliminar.
296  */
297 export function cerrarReporte(reporteId: string): ReporteMensual | null {
298   const all = readReportes();
299   const idx = all.findIndex((r) => r.id === reporteId);
300   if (idx < 0) return null;
301
302   const reporte = all[idx];
303   if (reporte.estado === "cerrado") return reporte; // ya cerrado
304
305   all[idx] = {
306     ...reporte,
307     estado: "cerrado",
308     cerrado_en: new Date().toISOString(),
309   };
310
311   writeReportes(all);
312   return all[idx];
313 }
314
315 /**
316  * Elimina un reporte en estado borrador.
317  * Los reportes cerrados no se pueden eliminar.
```

```
 */
export function eliminarBorrador(reporteId: string): boolean {
  const all = readReportes();
  const idx = all.findIndex((r) => r.id === reporteId);
  if (idx < 0) return false;

  const reporte = all[idx];
  if (reporte.estado === "cerrado") return false;

  all.splice(idx, 1);
  writeReportes(all);
  return true;
}

/** Exporta MESES para uso en componentes */
export { MESES };
```


Página 33 de 33